



Fundamentos da Programação Orientada a Objetos

UNIDADE 02

Classes e objetos

| Introdução

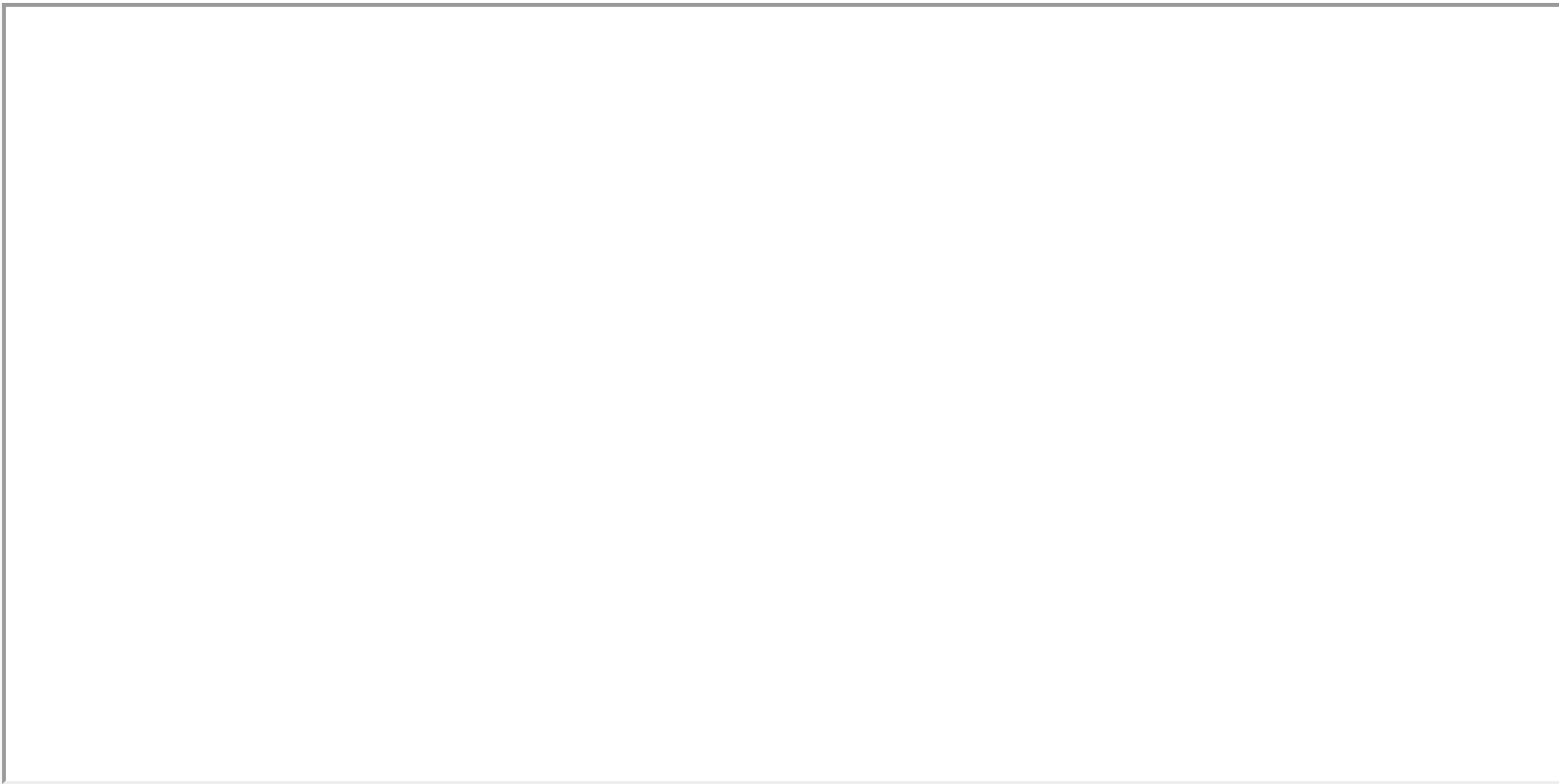
Olá!

Este será o momento de conversarmos sobre uma nova forma de desenvolvermos algoritmos: a Programação Orientada a Objetos (POO).

É provável que você tenha visto uma pequena introdução sobre este tema no final da disciplina Raciocínio Computacional. Dito isso, aprenderemos a partir de agora esse conceito usando Java.

Por isso, nesta unidade, aprenderemos a criar programas com as nossas primeiras classes e objetos, recursos essenciais nas linguagens orientadas a objetos, que permitem criar soluções computacionais de forma **intuitiva**.

Vamos lá?



Classe e objetos

Imagine uma criança brincando com blocos de construção (ou blocos de Lego). Cada bloco é único e tem suas próprias características, como cor, forma e tamanho. Agora, você pode juntar esses blocos de diferentes maneiras para criar coisas incríveis, como castelos, carros, animais e muito mais. Isso é muito parecido com como a maneira pela qual a Programação Orientada a Objetos (POO) funciona.

Na POO, tudo gira em torno de "**objetos**". Um objeto é como um bloco de construção. Ele contém informações (chamadas de "**atributos**" ou "**propriedades**") e coisas que ele pode fazer (chamadas de "**métodos**").

Por exemplo, pense em um objeto "Cachorro". Um cachorro tem atributos, como nome, cor e raça. Ele também tem coisas que pode fazer, ou seja, métodos, como latir, correr ou abanar o rabo.

Agora, de onde vem esse objeto "Cachorro"? Ele vem de algo chamado "**classe**". Uma classe é como um plano ou um modelo para criar objetos. A classe "Cachorro" define quais atributos e métodos um cachorro deve ter. Então, quando criamos um objeto a partir dessa classe, chamamos isso de "instância" da classe.

Não ficou claro? Tudo bem, vamos pensar em um caso baseado em empresas.

| Pensando com um exemplo real: Netflix

Bom, vamos pensar em um exemplo real? Vamos supor que você acabou de conseguir um emprego na Netflix (Disney+, GloboPlay, Prime Video ou qualquer outro serviço do gênero que você goste), e está trabalhando na equipe encarregada de desenhar a nova versão do aplicativo.

Agora, vamos pensar em como essa empresa funciona. Esqueça a programação: o que esta empresa **faz**? Como ela funciona? Talvez, poderíamos descrever algo como:

1. A Netflix tem vários **usuários**. Cada usuário tem informações como nome, e-mail e senha. Todos os usuários possuem essas características e podem fazer coisas parecidas, mas cada usuário é diferente dos demais.

- Exemplo: eu e você podemos ter contas separadas na Netflix. Eu tenho o meu e-mail de acesso, que é separado do seu. E, ainda, tenho a minha própria senha, que é diferente da sua.
- Eu e você também podemos fazer ações parecidas, como assistir a uma série, avaliar uma série, ou adicionar à lista de favoritos. Por outro lado, as séries que eu assisto podem ser completamente diferentes das séries que você assiste, não é?

2. A Netflix tem várias **séries**. Cada série tem as suas próprias informações como nome, gênero, classificação etária, número de temporadas e número de episódios.

- Exemplo: a série Stranger Things tem um número de temporadas e episódios diferente da série Brooklyn 99. Também existem outras informações que podem ajudar a diferenciar uma série da outra como, por exemplo, o gênero (ex.: comédia, ação, drama), o ano em que foi produzido e o próprio nome.

- Podemos interagir com uma série de formas diferentes. Os funcionários da Netflix podem optar por adicionar ou remover episódios específicos para o acesso do público, por exemplo. Os usuários da Netflix podem assistir ou avaliar as séries.

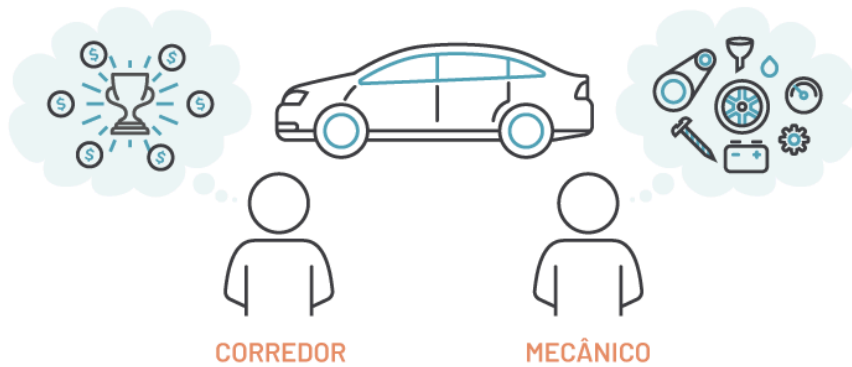
3. A Netflix também possui vários **episódios** das séries. Cada episódio também possui as suas informações como, por exemplo, o título, a duração em minutos e uma breve sinopse.

- Geralmente, podemos ver se já assistimos a um determinado episódio ou vemos em que ponto nós paramos em um determinado episódio, não é mesmo? Logo, podemos interagir com estes episódios com ações como “assistir” ou “marcar como assistido”, por exemplo.

O nome disso que acabamos de fazer é **abstração**: é uma maneira de pensarmos em como o mundo real funciona, mas de um jeito mais simplificado. Veja os exemplos a seguir: um mecânico pode entender um carro como sendo um conjunto de peças e componentes – já um piloto de corrida pode entender o mesmo carro como algo que o ajude a vencer corridas e ganhar prêmios. Um mapa nada mais é do que uma abstração do planeta como nós conhecemos, mas de um jeito bem mais simples.

Abstrair é observar um ou mais elementos de um todo, avaliando suas características e propriedades de forma isolada, com o objetivo de separar o elemento, ou o conceito, com o qual precisamos trabalhar, excluindo todos os demais. Aplicada à POO, a ação de abstrair consiste em separar conceitos e domínios do mundo real para representá-los como classes.

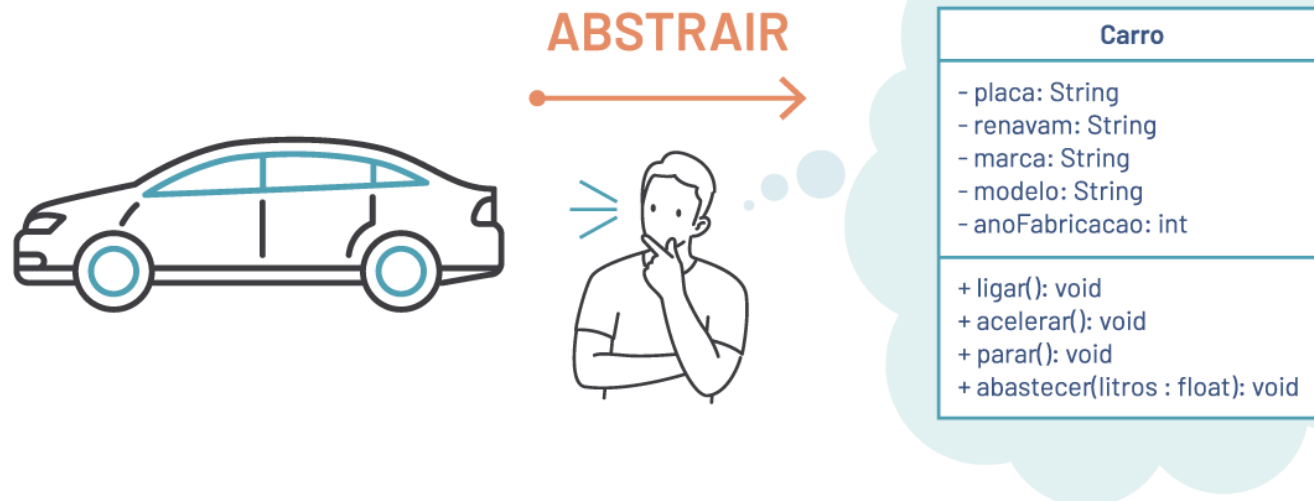
A abstração está nos olhos de quem vê...



Abstração



Abstração na POO



| Membros da classe: atributos e métodos

É a partir das abstrações das coisas do mundo real que poderemos pensar nos dois principais conceitos da POO: as classes e objetos, comparados na Figura a seguir.

Uma **classe** é como um modelo ou plano que descreve as propriedades e comportamentos comuns a um tipo de objeto, enquanto um **objeto** é uma instância (ou realização) desse plano, possuindo seu próprio conjunto de propriedades específicas e capaz de executar ações.

Em outras palavras, você pode pensar na classe como a ideia geral (ou a planta) de uma casa - sabemos que uma casa tem um certo número de quartos, banheiros, número de lâmpadas, pode estar em construção, ou estar pronta para morar, pode ter as suas luzes acesas ou apagadas, ou pode estar aberta ou trancada. Já o objeto é como uma casa em específico – é cinza, está construída e tem gente morando, está localizada na Rua São Pedro, 1000, está trancada no momento etc.

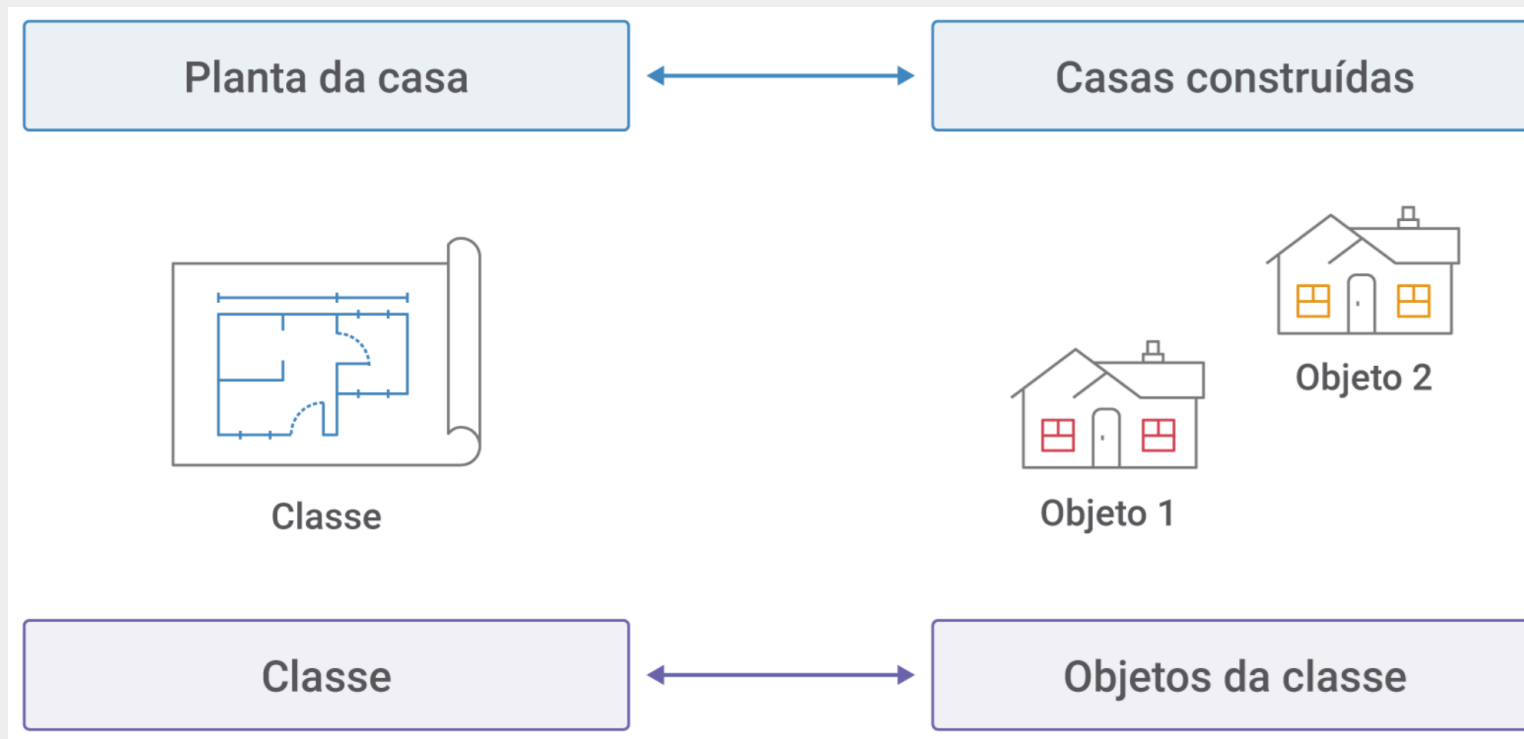


Figura 2: Classe x objetos. Fonte: Os autores (2021).

O que acha de pensarmos em alguns exemplos fazendo este mesmo exercício de abstração? A ideia é identificar os elementos que interessam para a construção de um determinado software.



DICA

Sabemos que programação orientada a objetos pode ser difícil de ser entendida no início. Afinal, é um outro jeito de programar. Por isso, sugerimos que use uma folha de papel para desenhar e replicar os exemplos: não falaremos de código, mas sim de conceitos. E é muito importante que você entenda estes conceitos antes de avançarmos com o Java.

Automação residencial: Lâmpadas em uma casa

Pense que precisamos desenvolver um programa para controlar as **lâmpadas de uma casa**. Na orientação a objetos, cada lâmpada é representada como um objeto, ou uma entidade, que tem um **estado** que pode estar “aceso” ou “apagado”, e os **comportamentos** de “acender a lâmpada” e “apagar a lâmpada”.

Essa modelagem inicial é mais bem-representada na orientação a objetos da seguinte forma:

- A abstração (ou ideia) de “lâmpada” é mapeada como uma classe.
- Qualquer classe tem uma estrutura composta por dois tipos de **membros**:
 - **Atributos**, ou estado da classe.
 - **Métodos**, ou comportamento da classe.
- As várias lâmpadas da casa – cada uma podendo estar acesa ou apagada – são as instâncias da classe, ou objetos da classe.

Para desenhar essas representações, utilizaremos uma linguagem de modelagem chamada UML.



DICA

UML significa "*Unified Modeling Language*" (linguagem de modelagem unificada, em português). É uma linguagem gráfica usada para visualizar, projetar e documentar sistemas de *software*. Ela fornece um conjunto de diagramas e notações que permitem descrever as diferentes perspectivas e aspectos de um sistema de forma clara e precisa.

A UML é amplamente utilizada no campo engenharia de *software* e desenvolvimento de sistemas como uma ferramenta-padrão para representar visualmente a estrutura, o comportamento, as interações e as relações entre os componentes de um sistema. Ela ajuda a comunicar e documentar o *design* e a arquitetura de um sistema de *software* de forma

visualmente compreensível, facilitando a colaboração entre os membros da equipe de desenvolvimento e permitindo uma melhor compreensão do sistema como um todo.

Por meio dos diagramas UML, os profissionais podem visualizar e comunicar de forma clara as relações entre as diferentes partes de um sistema, entender a interação entre os componentes, identificar requisitos, analisar o fluxo de informações e tomar decisões informadas durante o processo de desenvolvimento.

Profissionalmente falando, desenvolvedores raramente utilizam UML nas empresas para criar sistemas. Por outro lado, é comum encontrarmos esta notação quando gestores e arquitetos de sistema representam sistemas complexos.

Veja como fica a representação dessa modelagem, feita com UML, uma ferramenta de diagramação própria para a orientação a objetos, na figura a seguir.

Observe a representação de **classes** e **objetos** na notação do diagrama de classes da UML, ou linguagem unificada de modelagem.

#ParaTodosVerem 

Diagrama de classes da UML

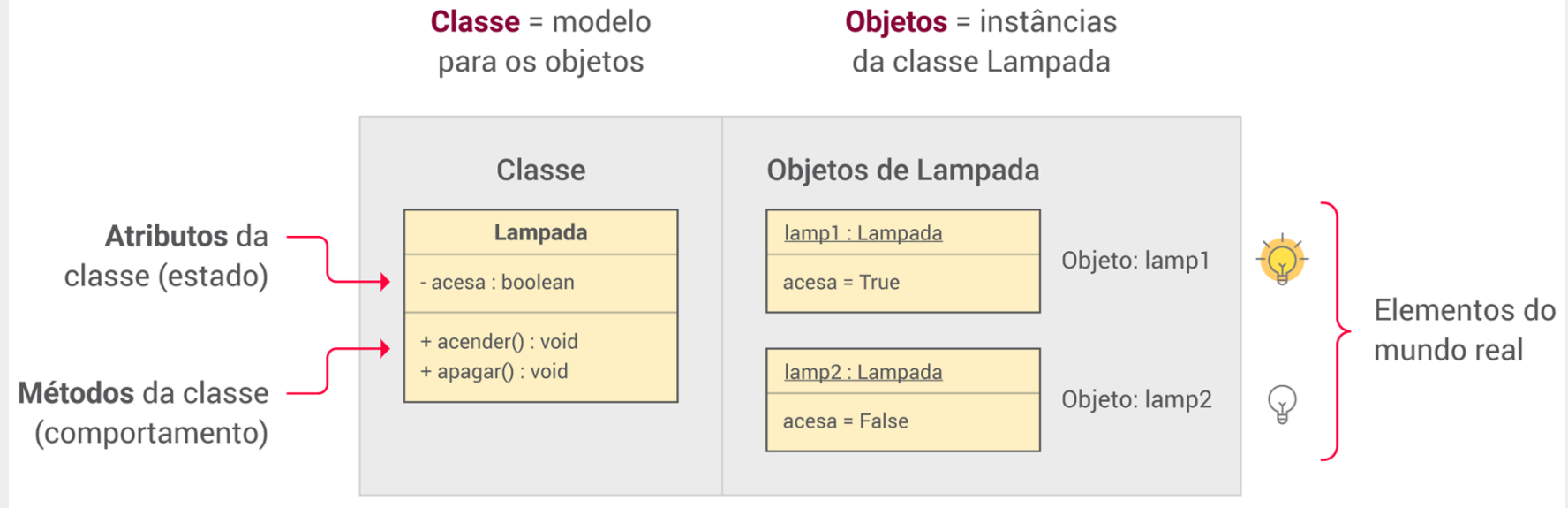


Figura 3: Pensando em classes e objetos. Fonte: Autores (2021).

O que conseguimos compreender do exemplo visual apresentado? Vejamos:

- Classe **Lampada**:
 - É a **representação** de qualquer lâmpada.
 - Toda lâmpada possui uma característica (estar acesa, ou não). Tais características possuem o nome de **atributos**.
 - Toda lâmpada possui ações/comportamentos: o de acender e o de apagar. Tais ações/comportamentos possuem o nome de **métodos**.
- Objetos:
 - A partir da classe Lampada, temos três objetos individuais, e um é independente do outro: **lamp1**, **lamp2** e **lamp3**.

- Cada um desses objetos pode estar em uma característica diferente dos demais (lamp1 está acesa, lamp2 está apagada, e lamp3 está acesa).
- As ações de acender e apagar ocorrem individualmente para cada uma dessas lâmpadas: por exemplo, ao apagar lamp1 gostaríamos de alterar o comportamento **somente** de lamp1, sem afetar os demais.

Vamos para mais um exemplo?

Netflix

Agora, como seria aquele exemplo da Netflix que pensamos anteriormente? Como seria possível abstrairmos a forma pela qual a Netflix funciona para criarmos algumas classes e objetos? Vejamos um exemplo:

1. Classe **Episodio**

- a. A Netflix possui uma lista de séries, e cada série possui vários episódios, não é? Logo, também poderíamos ter uma classe chamada **Episodio**. Esta classe possui **atributos** (características) que todo episódio deveria ter como, por exemplo, **título**, **duração** e **sinopse**.
- b. Todo episódio também possui os seus próprios **métodos** (ações). Isto inclui, por exemplo, **assistir**, **marcar como assistido**, ou **obter informações**.

2. Classe **Serie**

- a. Imagine que temos uma classe chamada **Serie**. Esta classe contém informações que toda série na Netflix tem. Logo, esta classe poderia possuir **atributos** (características) que toda série deveria ter, como **nome**, **gênero** e **classificação etária**.
- b. Toda série também possui **métodos** (ações) que clientes ou funcionários do Netflix possam realizar. Isto inclui, por exemplo, **adicionar episódios** e **remover episódios** da sua lista.

3. Classe **Usuario**

- a. A Netflix não existe sem os clientes – ou, ainda, usuários. Por isso, pode fazer sentido termos uma outra classe com o nome de **Usuario**. Todo usuário deve possuir os seus próprios **atributos** como, por exemplo, **nome**, **e-mail** e **senha de acesso**.

b. E, ainda, todo usuário pode interagir com a Netflix ao **assistir uma série, avaliar uma série, ou adicionar à sua lista de favoritos.**

A partir destas classes podemos ter seus objetos como, por exemplo:

1. Objetos de **Episodio**:

- a. **Título:** The End's Beginning, **Duração:** 61 minutos, **Sinopse:** "Following Geralt of Rivia's battle (...)"
- b. **Título:** Four Marks, **Duração:** 61 minutos, **Sinopse:** "In 1206, hunchback Yennefer from Vengerberg (...)"
- c. **Título:** Betrayer Moon, **Duração:** 67 minutos, **Sinopse:** "In 1210, Yennefer and Istredd become lovers (...)"
- d. **Título:** Chapter One: The Vanishing of Will Byers, **Duração:** 47 minutos, **Sinopse:** "At the U.S. Dept. of Energy an unexplained event occurs (...)"
- e. **Título:** Chapter Two: The Weirdo on Maple Street, **Duração:** 55 minutos, **Sinopse:** "Mike hides the mysterious girl in his house. Joyce gets a strange phone call."
- f. (...)

2. Objetos de **Serie**:

- a. **Nome:** Stranger Things, **Gênero:** Drama, **Classificação Etária:** 16, **Episódios:** [Chapter One: The Vanishing of Will Byers, Chapter Two: The Weirdo on Maple Street, ...]
- b. **Nome:** The Witcher, **Gênero:** Ação, **Classificação Etária:** 18, **Episódios:** [The End's Beginning, Four Marks, Betrayer Moon, ...]

3. Objetos de **Usuario**:

- a. **Nome:** "João", **E-mail:** "joao@gmail.com", **Senha:** "password123"
- b. **Nome:** "Marcela", **E-mail:** "marcela@outlook.com", **Senha:** "s3nh4000"

| Instância e referência de objetos

De forma resumida, podemos entender a **classe** como:

1. Pensando em analogias, imagine que a **classe** é como a receita de um bolo, que lista todos os ingredientes e etapas necessárias para fazer esse bolo. Você não pode comer a receita de bolo, mas ela te ensina como todo bolo deve ser, não é?
2. Ou, se você prefere um exemplo mais técnico, a **classe** é como um manual de instruções que descreve como criar e manipular um tipo específico de objeto, contendo todas as informações necessárias para que isso aconteça.
3. Ou, ainda, imagine a **classe** como sendo uma espécie de molde, onde você define características e comportamentos que serão comuns a todos os objetos que forem criados a partir dela.
4. Ou, sendo mais diretos e pensando em programação, pense na **classe** como uma forma de agrupar variáveis e funções relacionadas em um único pacote, com o objetivo de reutilizar código e facilitar a organização do seu programa.
5. Ou, de forma resumida, uma **classe** é como um projeto ou plano que define um tipo de **objeto**.

Agora, se uma classe é isso, o que seria um **objeto**? Bom, pensemos juntos:

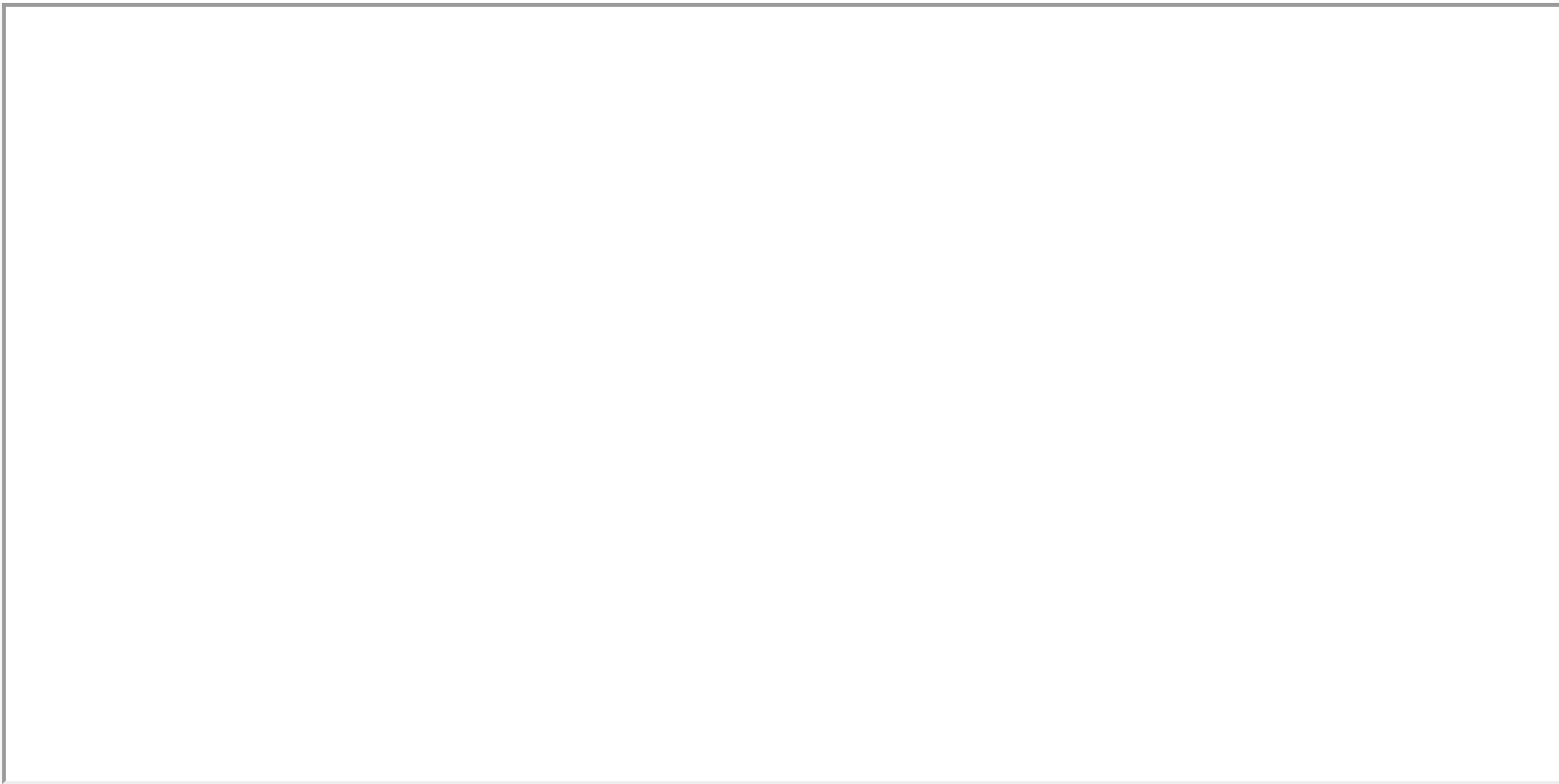
1. Se você pensar na **classe** como a receita de um bolo, um **objeto** seria o bolo que você fez seguindo as instruções da receita. Você pode criar vários bolos a partir da mesma receita, e cada bolo será diferente dos demais.
2. Ou, ainda, se a **classe** era o manual de instruções, o **objeto** é o produto final que você constrói seguindo essas instruções, cada um com suas próprias características específicas e capacidade de executar ações.
3. Ou, se a **classe** era um molde, um **objeto** é como um item feito a partir de um molde, possuindo todas as características e comportamentos definidos na classe, mas com seus próprios valores específicos.
4. Ou, pensando em programação, um **objeto** é um pacote de variáveis e funções que foram agrupadas de acordo com a definição de uma **classe**. Cada **objeto** é uma entidade única com suas próprias informações.
5. Ou, de forma bem resumida, um **objeto** é uma instância de uma classe, como uma casa construída a partir de um projeto ou plano.

Portanto, **objetos são criados a partir de uma classe**. Objetos precisam de uma classe para existir, e é aí que entra a **instanciação**. Pensemos no exemplo da classe como sendo uma receita do bolo. A receita contém todas as instruções e ingredientes necessários para fazer um bolo, mas a receita em si não é um bolo – é apenas o plano para fazer um.

A instanciação, neste caso, seria como seguir a receita para realmente fazer o bolo. Quando você instancia uma classe, está criando um objeto – um “bolo real” que você pode realmente comer. Cada objeto (ou bolo) que você cria é uma instância única da classe (ou receita) – pode seguir o mesmo conjunto básico de instruções, mas você pode personalizar detalhes específicos para cada um (talvez adicionar um pouco mais de chocolate em um, ou menos açúcar em outro).

RESUMO: Instanciação é o processo de criar um objeto (como um bolo) a partir de uma classe (como uma receita de bolo).

Criando e instanciando nossa primeira classe



No Java, cada classe é definida em um arquivo próprio, que tem o nome idêntico ao da classe e extensão .java. Como primeiro exemplo, iremos trabalhar com algumas formas geométricas. Vamos começar com um retângulo. Ele representa uma entidade de mundo real, logo pode ser representado como uma classe, mantida em um arquivo próprio. Depois disso, trabalharemos com exemplos mais complexos e mais próximos da vida real.



EXEMPLO

Exemplo de aplicação 1: crie um algoritmo em Java o qual possua uma classe chamada “Retangulo” (sem acentos no nome). Instancie um objeto para esta classe.

Como resolveremos isso? Bom, pensemos em como esta classe deverá funcionar:

- **Classe Retângulo**
 - Definido pelas suas medidas de “altura” e “largura” (atributos).
 - Um retângulo pode “calcular o seu perímetro” e “apresentar os seus dados” (métodos).

BOAS PRÁTICAS DE PROGRAMAÇÃO

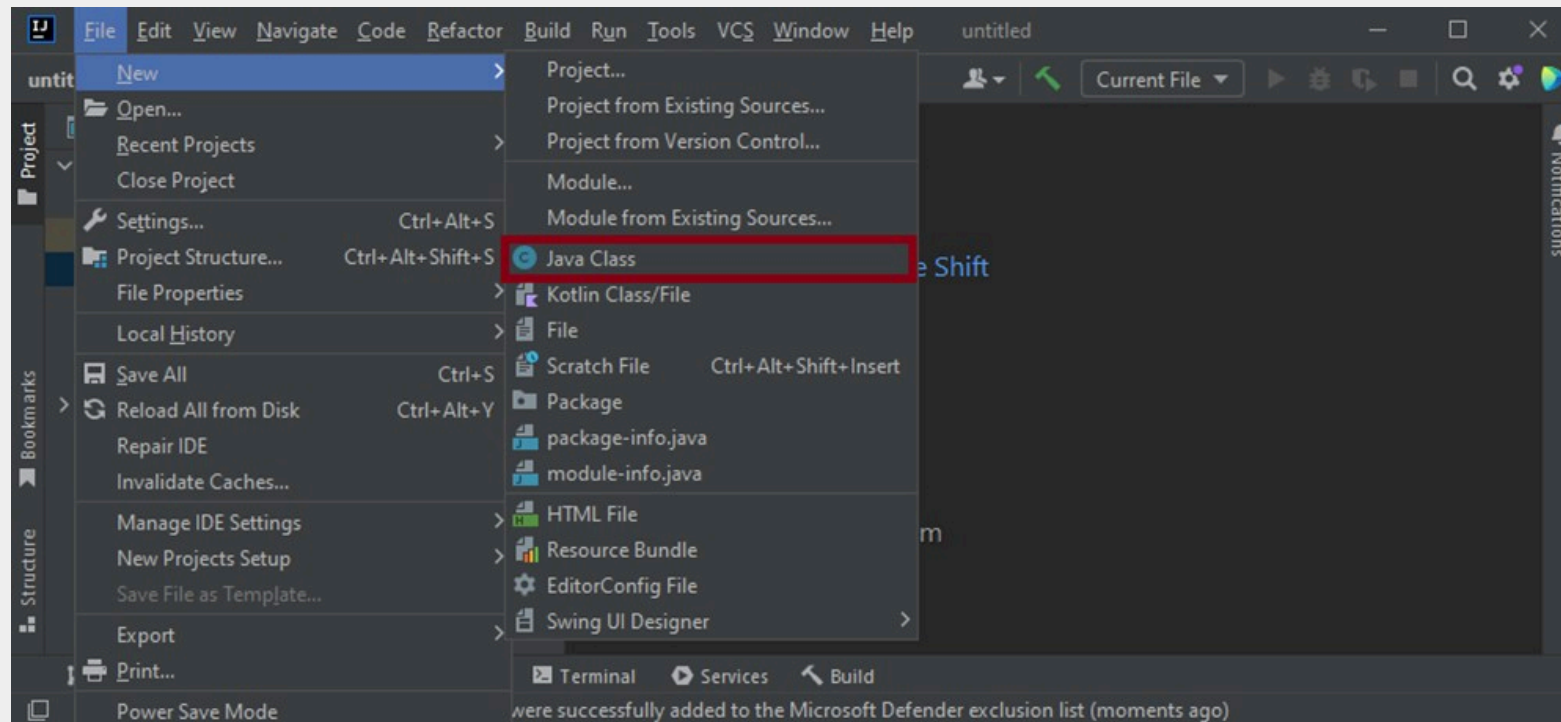
- Nome de classe = inicie com letra **maiúscula**.
- Nome de atributo = inicie com letra **minúscula** e use o nome de um **substantivo**.
- Nome de método = inicie com letra **minúscula** e use o nome de um **verbo**.
- Não use acentos da língua portuguesa em identificadores de qualquer linguagem de programação: nome de método, de classe ou de atributo.

Crie um projeto do IntelliJ (ou Eclipse, ou VS Code, ou qualquer outra IDE). Depois de criar, e se você estiver no IntelliJ, basta clicar em **File > New > Java Class**.



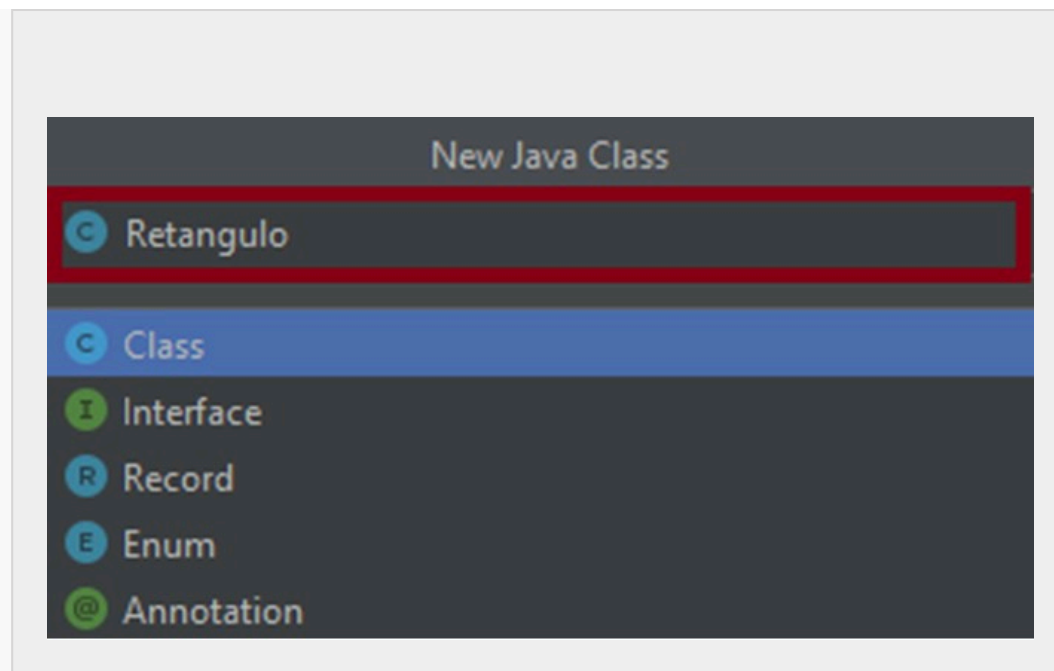
IMPORTANTE

Só aparecerá esta opção se você tiver selecionado a pasta src ou a pasta principal com o nome do seu projeto na lista de arquivos no menu da esquerda do IntelliJ.



Na tela que aparecer, basta digitar **Retangulo** como o nome da classe (sem acentos, e com a primeira letra maiúscula), e apertar **Enter**.

#ParaTodosVerem 



Agora, é hora de criar o código da classe. Copie e cole o código abaixo dentro do seu arquivo **Retangulo.java** que acabou de ser criado.

</>

```
1 public class Retangulo {
2     // Atributos da classe = ESTADO da classe
3     float altura;
4     float largura;
5
6     // Métodos da classe = COMPORTAMENTO da classe
7     float calcularPerimetro() {
8         float pm;           // variável local
9         pm = 2 * altura + 2 * largura;
10        return pm;
11    }
12    void imprimirDados() {
13        float p;             // variável local
14        p = calcularPerimetro(); // calcula o perímetro do retângulo
15        System.out.println("Retângulo: ");
16        System.out.println("- altura:    " + altura);
17        System.out.println("- largura:   " + largura);
18        System.out.println("- perímetro: " + p);
19    }
```

Depois disso, execute este mesmo arquivo usando as mesmas técnicas que aprendemos na semana passada. Você deverá visualizar um resultado parecido com este:

Mundo dos retângulos

Retângulo:

- altura: 10.0
- largura: 20.0
- perímetro: 60.0

Retângulo:

- altura: 5.0

```
- largura: 15.0  
- perimetro: 40.0
```

Process finished with exit code 0

Ao olhar o resultado deste código dá a entender que, aparentemente, dois retângulos foram criados. Estes retângulos possuem alturas e larguras diferentes. Como a altura e largura mudam, também muda o perímetro. Agora, como o código chegou a isso? Como aquele código que acabamos de colar funciona? Vamos entender agora, parte por parte:

Primeira parte: a definição da classe e dos seus atributos

```
</>  
  
1 public class Retangulo {  
2     // Atributos da classe = ESTADO da classe  
3     float altura;  
4     float largura;  
5     ...  
... }  
38
```

Esta primeira parte do código define uma nova classe chamada **Retangulo**. A classe tem dois atributos: altura e largura, ambos do tipo **float**. Perceba que as chaves que abrimos na linha 1 são fechadas na linha 38: isto significa que a definição da nossa classe ocupa 38 linhas de código, e tudo o que está dentro destas 38 linhas fazem parte da classe **Retangulo**.

Segunda parte: a definição dos métodos da classe

</>

```
6 // Métodos da classe = COMPORTAMENTO da classe
7 float calcularPerimetro() {
8     float pm; // variável local
9     pm = 2 * altura + 2 * largura;
10    return pm;
11 }
12 void imprimirDados() {
13     float p; // variável local
14     p = calcularPerimetro(); // calcula o perímetro do retângulo
15     System.out.println("Retângulo: ");
16     System.out.println("- altura: " + altura);
17     System.out.println("- largura: " + largura);
18     System.out.println("- perímetro: " + p);
19 }
```

As linhas 6 a 19 definem dois métodos para a classe **Retangulo**. O primeiro, **calcularPerimetro()**, calcula o perímetro de um retângulo (2 vezes a altura mais 2 vezes a largura) e retorna o resultado. O segundo, **imprimirDados()**, calcula o perímetro utilizando o método anterior e então imprime a altura, a largura e o perímetro do retângulo.



IMPORTANTE

Percebeu que antes do nome de cada função existe um tipo de dado? “**float calcularPerimetro()**” indica que esta função retorna um valor do tipo **float**. Se ela tivesse a definição “**int calcularPerimetro()**” ela deveria retornar um valor do tipo **int**. Se ela se chamasse “**String calcularPerimetro()**” ela deveria retornar um valor do tipo **String**. Agora, o que significa o **void** em “**void imprimirDados()**”? No caso, todas as funções que não retornam nada usam o **void**.

</>

```
20 // Método principal - início de execução do programa
21 public static void main(String[] args) {
22     System.out.println("Mundo dos retângulos");
23
24 ...
37 }
```

Em Java, precisamos ter uma porta de entrada do programa em algum lugar: caso contrário, ele não saberá exatamente onde (ou como) o seu algoritmo deve iniciar. Por isso, criamos um método chamado **main()**. Este é o método principal do programa, onde a execução começa. O primeiro comando imprime a frase “Mundo dos retângulos” na tela.



IMPORTANTE

Todo projeto em Java precisa ter pelo menos um método “*main*”. Você não precisa fazer isso para cada classe que você for criar, mas pelo menos em alguma parte do seu projeto um método chamado “*main*” precisa existir.

Quarta parte: instanciando e manipulando o primeiro objeto

</>

```
24      // Objeto de Retangulo retg1
25      Retangulo retg1;          // referência de objeto da classe Retangulo
26      retg1 = new Retangulo(); // instânciação da classe Retangulo em objeto
27      retg1.altura = 10;        // atribuição de valor para atributo do objeto
28      retg1.largura = 20;       // atribuição de valor para atributo do objeto
29      retg1.imprimirDados();    // invocação de método do objeto
30
```

Aqui, estamos criando um objeto da classe **Retangulo**, chamado **retg1**, por meio do processo de instânciação. Em seguida, definimos a altura e a largura desse retângulo e, finalmente, chamamos o método **imprimirDados()** para imprimir suas informações na tela.

Quinta parte: instanciando e manipulando o segundo objeto

</>

```
31      // Objeto de Retangulo retg2
32      Retangulo retg2;          // referência de objeto da classe Retangulo
33      retg2 = new Retangulo(); // instânciação da classe Retangulo em objeto
34      retg2.altura = 5;         // atribuição de valor para atributo do objeto
35      retg2.largura = 15;       // atribuição de valor para atributo do objeto
36      retg2.imprimirDados();    // invocação de método do objeto
37
```

Esta parte é muito semelhante à anterior. Estamos criando um segundo objeto **Retangulo**, chamado **retg2**, com uma altura e largura diferentes. Em seguida, chamamos o método **imprimirDados()** para imprimir suas informações na tela.

Veja que nas linhas 25 e 32, criamos uma variável do tipo Retangulo, mas ela ainda não está instanciada. A instanciação acontece nas linhas 26 e 33: momentos em que criamos um objeto com o uso da palavra-chave **new**. O **new** alocará um espaço de memória em seu computador para manter o objeto.

SUGESTÃO: o que acha de criar um terceiro objeto com o nome de **retg3** logo após a linha 36? Mude a altura e a largura deste terceiro retângulo, e teste

NOME DO ARQUIVO Java: terá o mesmo nome da classe, com extensão .java:

- `Retangulo.java`

PARA DEFINIR UMA CLASSE no Java:

- Usar as palavras-chave `public class` seguidas do nome da classe: `public class Retangulo`

PARA CRIAR UM OBJETO no Java:

- Usar a palavra-chave `new` da seguinte forma:
- Opção 1 (duas linhas):

```
Retangulo retg1;           // nome de referência de objeto
```

```
retg1 = new Retangulo();    // instanciação da classe Retangulo
```

- Opção 2 (uma linha só):

```
Retangulo retg1 = new Retangulo();
```

- A referência de objeto é o nome do objeto de uma determinada classe.
- No exemplo, `retg1` é um objeto da classe `retangulo`.

| Invocação de métodos dos objetos

Conforme vimos anteriormente, os métodos de uma classe refletem o comportamento que o objeto pode realizar, ou seja, **operações** que o objeto pode executar. Por isso, é recomendado, sempre que possível, utilizar verbos para o nome do método, para indicar uma ação.

Um **método**, na orientação a objetos, equivale a uma **função** na programação procedural. O que aprendemos em Raciocínio Computacional era programação procedural.

A declaração de uma função segue este padrão:

```
tipo nomeMetodo (tipo parametro1, tipo parametro2, ..., tipo parametroN) {
```

```
// corpo do método
```

```
}
```

O tipo de dado de retorno do método, seguido do nome do método e da sua lista de parâmetros, define a assinatura do método. A assinatura de um método também pode ser entendida como sendo a identidade do método, incluindo seu nome e a lista de tipos de parâmetros que ele recebe, que juntos ajudam a identificar e diferenciar um método de todos os outros em uma classe. Cada parâmetro do método deve ter um tipo de dado declarado, como mostrado anteriormente.

Quando o método não tem retorno, é usada a palavra-chave **void**. Se o método não tem parâmetros, sua lista de parâmetros fica vazia.

O corpo do método define um bloco de código, delimitado por { e }, que pode usar as operações e os atributos da classe, mas também pode usar novas variáveis, que apenas existem dentro do método e são chamadas de variáveis locais.

Veja o exemplo a seguir do **imprimirDados ()**, que é um método sem lista de parâmetros e sem retorno.

</>

```
1 void imprimirDados() {  
2     float p;           // variável local  
3     p = calcularPerimetro(); // calcula o perímetro do retângulo  
4     System.out.println("Retângulo: ");  
5     System.out.println("- altura:    " + altura);  
6     System.out.println("- largura:   " + largura);  
7     System.out.println("- perímetro: " + p);
```

O método **imprimirDados ()** também invoca outro método, quando executa o comando:

```
p = calcularPerimetro(); // calcula o perímetro do retângulo
```

“**Invocar um método**”, na orientação a objetos, equivale a “**chamar uma função**” na programação procedural.

Quer ver alguns exemplos adicionais de invocação? No exemplo do código do retângulo, temos invocação de método nas **linhas 14, 29 e 36**.

A palavra-chave “**return**”

Quando um método tem um tipo de retorno, é obrigatório utilizar o comando Java **return** ao final da execução do método. É dessa forma que o retorno do método é atribuído à uma variável. Veja o exemplo do método **calcularPerimetro ()** do código do exercício dos retângulos.

</>

```
1 float calcularPerimetro() {  
2     float pm;           // variável local  
3     pm = 2 * altura + 2 * largura;  
4     return pm;  
5 }
```

Quando **calcularPerimetro ()** é invocado, ele atribui o seu retorno à uma variável do tipo **float**.

</>

```
1 float p;                // variável local p do tipo float  
2 p = calcularPerimetro(); // calcula o perímetro do retângulo e atribui à p
```

| Métodos construtores

Quando usamos o comando Java **new** para instanciar uma classe, ou seja, para criar um objeto, estamos usando um método especial chamado de **método construtor** (GODOY, 2019).

Na POO, um método construtor é um tipo especial de método que é usado para inicializar um objeto recém-criado. No Java, quando usamos o comando Java **new** para instanciar uma classe, estamos utilizando construtores.

Vamos pensar em analogias para entendermos melhor os construtores. Sendo assim, imagine a montagem de um brinquedo a partir de um kit complexo de peças. Assim, imagine que você comprou um kit de construção de um carro de brinquedo. Esse kit vem com todas as peças que você precisa para montar o carro, mas ainda não é um carro completo.

Nesse caso, o construtor seria como o manual de instruções que acompanha o kit. Ele guia você passo a passo na montagem das peças, para que você termine com um carro de brinquedo completo. Da mesma forma, na programação, quando você cria um objeto (ou seja, quando você "compra o kit"), o construtor é o código que é executado automaticamente para configurar esse objeto corretamente (ou seja, "montar o carro").

Pensemos novamente no nosso código do retângulo: se você tem uma classe **Retangulo** com atributos como **altura** e **largura**, o construtor da classe **Retangulo** pode ser usado para definir a altura e largura automaticamente toda vez que você cria um retângulo.

Todas as classes têm construtores, mesmo quando não estão definidos no código, pois o Java fornece automaticamente um construtor-padrão que inicializa os atributos da classe para os seus valores-padrão: zero para valores numéricos, false para valores booleanos e null para os demais tipos de dados.

O que acha de adaptarmos o código dos retângulos para usar construtores?



EXEMPLO

Exemplo de aplicação 2: adapte a classe **Retangulo** para que ela contenha um construtor o qual aceite dois parâmetros: a altura e a largura.

Agora, substitua o código do arquivo **Retangulo.java** por este:

```
</>
```

```
1 public class Retangulo {
2     // Atributos da classe = ESTADO da classe
3     float altura;
4     float largura;
5
6     public Retangulo (float alt, float larg) { // Define o método construtor
7         altura = alt;
8         largura = larg;
9     }
10    // Métodos da classe = COMPORTAMENTO da classe
11    float calcularPerimetro() {
12        float pm;                // variável local
13        pm = 2 * altura + 2 * largura;
14        return pm;
15    }
16    void imprimirDados() {
17        float p;                // variável local
18        p = calcularPerimetro(); // calcula o perímetro do retângulo
19        System.out.println("Retângulo: ");
```

Agora, o que mudou? Perceba principalmente as linhas 6 a 9, 30 e 35. Em suma:

1. Linhas 6-9: agora, temos um método construtor criado. Ele aceita dois parâmetros: “**alt**” (que corresponderá à **altura** definida na linha 3), e “**larg**” (que corresponderá à **largura** definida na linha 4).
2. Linhas 30 e 35: em vez de “`retg1 = new Retangulo()`” e “`retg2 = new Retangulo()`” e termos de definir a altura e largura depois, agora, já definimos a altura e a largura em uma única linha.
 - a. Agora, precisaremos sempre criar os retângulos seguindo exatamente a ordem estabelecida pelo nosso construtor: sempre dois parâmetros, e sempre a altura seguida pela largura.



IMPORTANTE

Perceba duas coisas aqui. Primeiramente: o nome do construtor possui **o mesmo nome da sua classe**. Segundo: a ordem dos parâmetros do construtor precisará ser obedecida. Se no nosso código definimos que primeiro vem a altura e depois vem a largura, então todas as chamadas desse construtor deverão informar a altura primeiramente e, depois, a largura.

| Palavras-chave do Java

Profissionais que atuam com programação precisam também saber o que são (e como funcionam) palavras-chave. As palavras-chave são termos pré-definidos que possuem um **significado especial** para o compilador Java. Elas definem a estrutura e o controle de como o código será executado.

Por exemplo, palavras-chave como **public**, **class**, **if**, **for**, e **return** desempenham papéis fundamentais na definição da visibilidade de uma classe ou método, na criação de uma classe, na tomada de decisões, na repetição de uma tarefa e na devolução de um valor de um método, respectivamente. Sem um bom entendimento das palavras-chave, você não poderá criar programas eficazes e eficientes em Java.

Vamos ver como o Java trata com alguns aspectos importantes da POO, utilizando palavras-chave da linguagem.

A palavra-chave "*this*"

Como os construtores servem para inicializar os atributos da classe, uma prática comum é declarar o nome dos parâmetros exatamente igual ao nome dos atributos nas classes. Só que isso tem um problema: o código escrito desse jeito gera ambiguidade. Veja este exemplo:

```
</>
```

```
1 public class Retangulo {
2     // Atributos da classe = ESTADO da classe
3     float altura;
4     float largura;
5
6     public Retangulo (float altura, float largura) { // Define o método construtor
7         altura = altura;
8         largura = largura;
9     }
10
11 }
```

Veja a linha 7. O que esta linha está querendo dizer? Será que estamos salvando o valor da variável “altura” da linha 3 na “altura” da linha 6? Ou a “altura” da linha 6 na linha 3? Ou algo totalmente diferente disso? Bom, se você não sabe, o Java **também** não saberia. Logo, temos aqui um problema bem grande de ambiguidade. O mesmo se aplica à largura da linha 8.

Para resolver isso, a palavra-chave **this** em Java é usada como uma referência à instância atual de um objeto. Ela é especialmente útil quando precisamos distinguir entre campos de classe e parâmetros de método que têm o mesmo nome. Por exemplo, se você tiver um método definido em uma classe que configura um campo da classe e o parâmetro do método tiver o mesmo nome que o campo, você pode usar **this** para se referir especificamente ao campo da classe.

Além disso, **this** também pode ser usado para chamar um construtor a partir de outro construtor dentro da mesma classe ou para passar a instância atual do objeto como um parâmetro para outro método. Veja o código modificado a seguir:

```
</>
```



```
1 public class Retangulo {
2     // Atributos da classe = ESTADO da classe
3     float altura;
4     float largura;
5
6     public Retangulo (float altura, float largura) { // Define o método construtor
7         this.altura = altura;
8         this.largura = largura;
9     }
10
11 }
```

Percebe o “this” nas linhas 7 e 8? A palavra-chave **this** é usada para se referir à instância atual do objeto. Então, quando dizemos “**this.altura = altura;**”, estamos dizendo “pegue a **altura** que foi passada para o construtor (linha 6) e use-a para definir a altura do retângulo que estamos construindo (linha 3)”. O mesmo vale para “**this.largura = largura;**”.

A palavra-chave “*static*”

O termo **static** em Java é uma palavra-chave que pode ser aplicada a variáveis, métodos, blocos de código e classes aninhadas. Vamos pensar no exemplo de uma calculadora para entender melhor.

Imagine uma calculadora científica que tem uma série de funções matemáticas embutidas. Algumas dessas funções, como **sqrt** (square root, ou raiz quadrada) ou **log** (logaritmo), **não dependem do estado atual da calculadora**. Ou seja, não importa o que foi calculado antes, o resultado dessas operações depende apenas dos valores atuais que você passa para elas. Essas funções poderiam ser representadas como métodos estáticos na classe **Calculadora**, porque **elas não precisam de um objeto de calculadora específico para funcionar**.

Vamos pensar em como isso seria em Java?



EXEMPLO

Exemplo de aplicação 3: crie uma classe chamada **Calculadora**, a qual possua dois métodos estáticos.

Para resolver isso, crie um arquivo chamado **Calculadora.java**. Cole o código a seguir neste arquivo, e execute o código:

</>

```
1 public class Calculadora {
2     public static double sqrt(double valor) {
3         return Math.sqrt(valor);
4     }
5
6     public static double log(double valor) {
7         return Math.log(valor);
8     }
9
10    public static void main(String[] args) {
11        int x = 25;
12        int y = 100;
13
14        double raiz = Calculadora.sqrt(x);
15        double logaritmo = Calculadora.log(y);
16
17        System.out.printf("Raiz quadrada de %d: %f \n", x, raiz);
18        System.out.printf("Logaritmo de %d: %f \n", y, logaritmo);
19    }
```

Agora, o que este código faz? Temos uma classe chamada **Calculadora** (linha 1) que tem dois métodos estáticos, **sqrt** e **log** (linhas 2 a 8). Esses métodos realizam a operação de raiz quadrada e logaritmo, respectivamente.

A palavra-chave **static** significa que esses métodos pertencem à classe **Calculadora** em si, não à uma instância específica (ou objeto) da classe. Em outras palavras, não precisamos criar um objeto **Calculadora** para usar esses métodos – podemos simplesmente chamá-los diretamente na classe. Percebe que não temos nenhuma instanciação de objetos de **Calculadora** nesse código?

No método **main**, que é o ponto de entrada do programa (onde a execução começa), criamos duas variáveis, **x** e **y**, e então, usamos os métodos estáticos **sqrt** e **log** da classe **Calculadora** para calcular a raiz quadrada de **x** e o logaritmo de **y**, respectivamente. Os resultados desses cálculos são armazenados nas variáveis **raiz** e **logaritmo**. Nos dois casos, a invocação desses métodos usa no nome da classe, e não o nome de um objeto, da classe **Calculadora** – invocações realizadas nas linhas 14 e 15.



DICA

Em Java, quando usamos **static**, estamos dizendo que aquele atributo ou método pertence à classe toda, e não somente a um objeto único em específico a partir dela.

Quando declaramos um atributo como **static**, ele é compartilhado por todas as instâncias da classe: o atributo é mantido na mesma área de memória para todas as instâncias da classe. Isso significa que se um objeto da classe alterar o valor do atributo, todos os demais objetos da mesma classe terão o mesmo valor alterado no seu atributo.

Definindo constantes com "*final static*"

É comum utilizar a palavra-chave **static** junto com a palavra-chave **final** (que não permite alteração) para definir constantes. Vamos para um exemplo?

Exemplo de aplicação 4: crie uma classe chamada **Circulo**. Essa classe precisa ter um método encarregado de calcular o perímetro do círculo. O perímetro de um círculo segue a equação *null*

Para resolver isso, crie um arquivo chamado **Circulo.java**. Cole o código abaixo neste arquivo, e execute o código:

```
</>
1  public class Circulo {
2      // Atributos da classe
3      double raio;
4
5      // Constante PI definida como final static
6      final static double PI = 3.141592653589793;
7
8      // Método construtor
9      public Circulo(double raio) {
10         this.raio = raio;
11     }
12
13     // Método para calcular o perímetro (ou circunferência) do círculo
14     public double calcularPerimetro() {
15         return 2 * PI * this.raio;
16     }
17
18     // Método principal
19     public static void main(String[] args) {
```

Neste exemplo, a instância **meuCirculo** de **Circulo** é criada com um raio de 5 (linha 21). Usamos o método **calcularPerimetro** para calcular o perímetro desse círculo e, em seguida, imprimimos o resultado.

Agora, veja a linha 6: usamos o **final static** definir **PI**, que é uma constante que todos os círculos compartilham (afinal, o valor de pi não muda, não é?). A palavra-chave **final** significa que o valor de PI não pode ser alterado após ser definido, e **static** significa que PI pertence à classe **Circulo**, e não a um objeto específico dessa classe. Portanto, qualquer objeto **Circulo** que criarmos terá acesso a esse mesmo valor de PI.

O método **calcularPerimetro** (linhas 14-16) calcula o perímetro (ou circunferência) do círculo, que é $2 * PI * \text{raio}$. Ele usa o valor de PI que definimos anteriormente e o raio que é específico para o objeto **Circulo**.



DICA

Usamos **final static** quando temos uma constante que todos os objetos da classe compartilham e que não deve ser alterada.

| Trabalhando com listas de objetos

Até agora, vimos como criar um objeto isolado, como uma lâmpada ou um retângulo. Mas no mundo real, e no nosso exemplo da Netflix, as coisas andam em grupos: uma *Série* tem vários *Episódios*, um *Usuário* tem várias *Séries* assistidas.

No Java, a maneira mais fácil de lidar com isso não é criando variáveis como “episodio1”, “episodio2”, “episodio3”, e assim por diante. Na prática, isto seria impossível de gerenciar. É por isso que nós usamos uma estrutura chamada *ArrayList*. Ela lembra um pouco a estrutura de listas que vimos anteriormente na disciplina de Raciocínio Computacional. Isto será útil para os nossos exercícios de fixação, abaixo.

Vejamos um exemplo que usa isto abaixo. Perceba que na linha 24 instanciamos o nosso *ArrayList* vazio, e ele aceita somente objetos do tipo “*Episodio*”. Já nas linhas 32 a 34 adicionamos 3 episódios à lista (e poderíamos adicionar vários outros se quiséssemos).

Ainda, veja que na linha 46 temos uma forma um pouco diferente de percorrer todos os objetos de uma lista: esta variante do “for” se chama “for each”. A sua sintaxe é um pouco mais simples, e a interpretamos como “para cada item dessa lista, faça:”. O que acha de testar o código abaixo e debugar para entender melhor o que faz cada linha?

```
</>
1  import java.util.ArrayList; // Importamos a ferramenta de Lista do Java
2
3  class Episodio {
4      String titulo;
5      int duracao;
6
7      // Construtor
8      public Episodio(String titulo, int duracao) {
9          this.titulo = titulo;
10         this.duracao = duracao;
11     }
12
13     // Método para mostrar os dados na tela
14     public void imprimirInfo() {
15         System.out.println("Episódio: " + this.titulo + " (" + this.duracao + " min)");
16     }
17 }
18
19 public class Main {
```

| Exercícios de fixação

O que acha de fazermos alguns exercícios? Todos eles usarão o exemplo da Netflix como base. Se você já possui experiência na área, perceberá que não estamos ainda trabalhando com conceitos de acesso (*public/private*): veremos sobre isso em breve.



DICA

Os exercícios podem rapidamente aumentar em sua complexidade. Não se sinta apreensivo caso não consiga fazer por conta própria algum exercício. Procure analisar a solução e transcrever as linhas – retome os conceitos que vimos anteriormente como construtores, métodos e atributos. Procure entender o que cada bloco de código faz e, se assim preferir, inclua um comentário a cada linha ou bloco de código com o seu entendimento. Lembre-se: aprendemos com a experiência e colocando a mão na massa. Ah, e lembre-se também: existe sempre o **Fale com a Gente** para te ajudar!



EXERCÍCIO

Nível fácil

1. Desenvolva um projeto em Java contendo somente um arquivo: **Usuario.java**. Esse arquivo deve conter uma classe chamada **Usuario**, que representa um usuário da Netflix. Essa classe precisa de três atributos (nome do usuário, e-mail e senha), somente.
2. Modifique o exercício anterior para que ele contenha dois métodos (um para mostrar na tela o nome e e-mail do usuário, e outro para verificar se uma determinada senha informada é igual à senha cadastrada pelo usuário). Além disso, inclua um método construtor para aceitar três parâmetros: o e-mail, a senha e o nome do usuário.
3. Modifique o exercício anterior para instanciar um objeto para a classe **Usuario**, representando um usuário com o nome de “João”, e-mail “joao@exemplo.com”, e senha “senhaSegura123”. Chame o método “verificarSenha” duas vezes: uma utilizando a senha correta (“senhaSegura123”) e outra utilizando uma senha incorreta (exemplo: “senhaSegura000”).

Nível médio

4. Crie uma classe `ContaBancaria` que tenha um atributo “saldo” (double) e um atributo “titular”. Esse atributo “titular” não deve ser um texto simples, mas sim um objeto da classe `Usuario` (que você criou no nível fácil). No método principal, crie um `Usuário`, depois crie uma `Conta` e atribua esse usuário a ela.
5. Modifique o exercício 3 para adicionar um novo arquivo no projeto com o nome de **Episodio.java**. Este arquivo deve conter uma classe chamada **Episodio**, o qual possui três atributos (título e duração), e um método que mostra na tela esses dois atributos. Instancie cinco objetos desta classe.

Nível difícil

6. Modifique o exercício anterior para adicionar um novo arquivo no projeto com o nome de **Serie.java**. Este arquivo deve conter uma classe chamada **Serie**, o qual possui dois atributos: o nome da série e uma lista de episódios (objetos do tipo **Episodio** que criamos no exercício anterior). Esta classe precisa de um método que mostre na tela os dados da Série.

Desafio

7. Modifique o arquivo **Usuario.java**. Agora, a classe **Usuario** precisa de um novo atributo chamado **seriesAssistidas**. Ele deve ser uma lista de séries que o usuário já assistiu. Também, adicione um método chamado “adicionarSerieAoHistorico” para que consigamos adicionar novas séries ao histórico do usuário. Finalmente, modifique o método encarregado por mostrar as informações do usuário para que mostre também as séries que ele assistiu.



Resolução do exercício



Exercício 1:

```
</>
```



```
1 public class Usuario {  
2     // Atributos da classe  
3     String nome;  
4     String email;  
5     String senha;  
6  
7     // Método principal  
8     public static void main(String[] args) {  
9  
10    }  
11 }
```

Exercício 2:

</>

```
1 public class Usuario {
2     // Atributos da classe
3     String nome;
4     String email;
5     String senha;
6
7     // Método construtor
8     Usuario(String nome, String email, String senha) {
9         this.nome = nome;
10        this.email = email;
11        this.senha = senha;
12    }
13
14    // Método para imprimir as informações do usuário
15    void imprimirInfo() {
16        System.out.println("Nome: " + this.nome);
17        System.out.println("Email: " + this.email);
18    }
19
```

Exercício 3:

</>

```
1 public class Usuario {
2     // Atributos da classe
3     String nome;
4     String email;
5     String senha;
6
7     // Método construtor
8     Usuario(String nome, String email, String senha) {
9         this.nome = nome;
10        this.email = email;
11        this.senha = senha;
12    }
13
14    // Método para imprimir as informações do usuário
15    void imprimirInfo() {
16        System.out.println("Nome: " + this.nome);
17        System.out.println("Email: " + this.email);
18    }
19 }
```

Exercício 4:

</>

```
1 // 1. Classe Usuario (que você já fez no nível fácil)
2 class Usuario {
3     String nome;
4     String email;
5     String senha;
6
7     public Usuario(String nome, String email, String senha) {
8         this.nome = nome;
9         this.email = email;
10        this.senha = senha;
11    }
12 }
13
14 // 2. Classe ContaBancaria (O novo conceito)
15 class ContaBancaria {
16     double saldo;
17     Usuario titular; // veja que o tipo do atributo é a classe Usuario!
18
19     // No construtor, pedimos um objeto Usuario inteiro, não só o nome
```

Exercício 5:

</>

```
1 public class Episodio {
2     // Atributos da classe
3     private String titulo;
4     private int duracao;
5
6     // Método construtor da classe Episodio
7     public Episodio(String titulo, int duracao) {
8         this.titulo = titulo;
9         this.duracao = duracao;
10    }
11
12    // Método para imprimir as informações do episódio
13    public void imprimirInfoEpisodio() {
14        System.out.println("Titulo: " + this.titulo);
15        System.out.println("Duração: " + this.duracao + " minutos");
16    }
17
18    // Método principal
19    public static void main(String[] args) {
```

Exercício 6:

</>

```
1 import java.util.ArrayList;
2
3 public class Serie {
4     // Atributos da classe Serie
5     String nome;
6     ArrayList<Episodio> episodios;
7
8     // Método construtor da classe Serie
9     Serie(String nome) {
10         this.nome = nome;
11         this.episodios = new ArrayList<>();
12     }
13
14     // Método para adicionar episódio à série
15     void adicionarEpisodio(Episodio episodio) {
16         this.episodios.add(episodio);
17     }
18
19     // Método para imprimir as informações da série
```

Exercício 7:

</>

```
1 import java.util.ArrayList;
2
3 public class Usuario {
4     // Atributos da classe
5     String nome;
6     String email;
7     String senha;
8     ArrayList<Serie> seriesAssistidas;
9
10    // Método construtor
11    Usuario(String nome, String email, String senha) {
12        this.nome = nome;
13        this.email = email;
14        this.senha = senha;
15        this.seriesAssistidas = new ArrayList<>(); // iniciando uma lista vazia
16    }
17
18    // Método para imprimir as informações do usuário
19    void imprimirInfo() {
```

| Considerações finais

Nesta Unidade, aprendemos que uma classe é a representação de algo no mundo real: uma entidade ou um conceito. Uma classe tem determinada estrutura especial para manter seus membros: atributos e métodos.

- Os atributos definem o que precisamos manter do objeto: são seus dados, suas variáveis de classe, ou são o **estado** da classe.
- Os métodos definem as ações ou operações que os objetos de uma classe podem realizar; são o **comportamento** da classe.

Aprendemos também que criamos objetos quando instanciamos uma classe. Cada um mantém um estado próprio, e todos os objetos de uma classe podem executar as mesmas operações ou métodos.

Vimos também que uma classe possui um método especial, chamado quando desejamos criar o objeto da classe: o método construtor, usado para inicializar os atributos da classe. No Java, para invocar o método construtor que instancia sua classe, usamos a palavra-chave **new**.

Por fim, aprendemos que vários aspectos da orientação a objetos são tratados no Java com algumas palavras-chave da linguagem, como:

- **this**: referência do objeto para si próprio.
- **static**: precede o nome de atributos e métodos da classe que são utilizados sem a necessidade de instanciar a classe (ou criar objetos da classe); são atributos e métodos de classe, e não de objeto:
 - Atributo **static**: atributo de classe, cujo valor é o mesmo para todas as instâncias da classe; uma vez alterado, todas as instâncias da classe visualizam o mesmo novo valor.
 - **Final static**: permite a criação de atributos de classe que são constantes, cujos valores não podem ser alterados.

| Referências Bibliográficas

GODOY, V. **Programação orientada a objetos I**. Curitiba, PR: IESDE, 2019.

HORSTMANN, C. S.; CORNELL, G. **Core Java**. 8. ed. São Paulo, SP: Pearson, 2010.

SCHILDT, H. **Java para iniciantes**. Porto Alegre, RS: Bookman, 2015.



© PUCPR - Todos os direitos reservados.